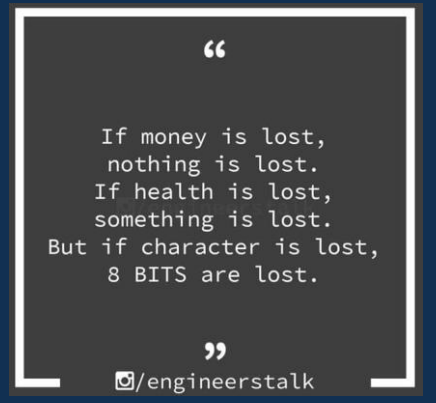
03. Strings

Kyu-haeng Lee

SW Convergence Coding II

Fall 2025



DKU CI Lab

String Literals

String Literals

- String Literal
  - A sequence of characters enclosed within **“double quotes”**
  - ex) “Dankook University”

- String literals may contain escape sequences.
  - Each \n character causes the cursor to advance to the next line
  - “Dankook\nUniversity\n”

  - Dankook
  - University

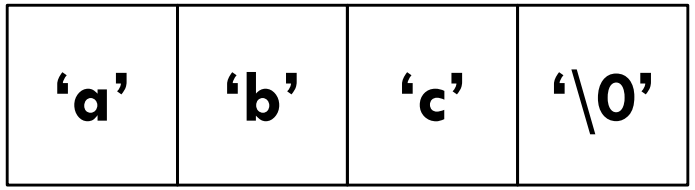
Continuing a String Literal

- The backslash character (\)
  - Used to join two or more lines of a program into a single line.
  - printf("When you come to a fork in the road, take it. \
    --Yogi Berra");

- When two or more string literals are adjacent, the compiler will join them into a single string.
  - printf("When you come to a fork in the road, take it."
    "--Yogi Berra");
  - This rule allows us to split a string literal over two or more lines

How String Literals Are Stored

- A string literal is stored as an array
  - For “abc”

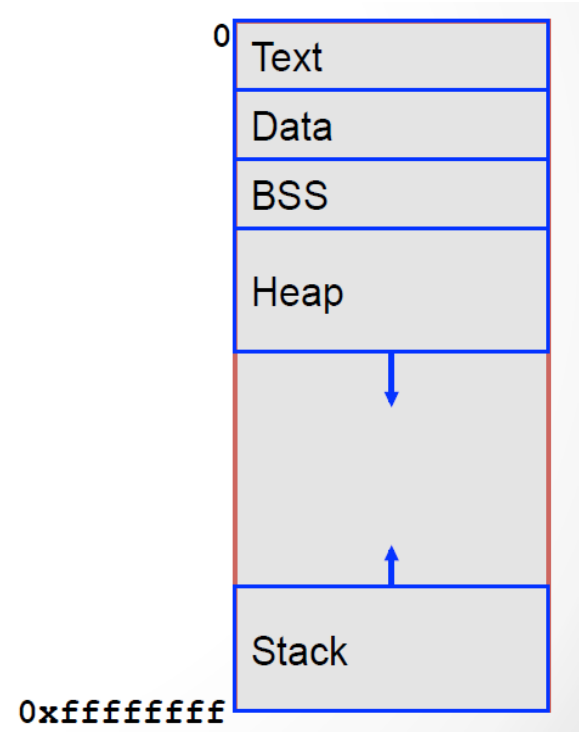


  - For “”, a single null character ‘\0’ is stored.

- Since a string literal is an array, C treats it as a pointer.

How String Literals Are Stored

- String literals are stored
  in the text area of memory,
  where read-only operations are available.



- Attempting to modify a string literal causes undefined behavior:
  - char *p = "abc";
  - *p = 'd'; /*** WRONG ***/

How String Literals Are Stored

- Example
  - Both printf and scanf expect a value of type char * as their first argument.

  - printf("abc");
    - printf passes the address of "abc" (a pointer to where the letter a is stored in memory):

Operations on String Literals

- Can use a string literal wherever C allows a char * pointer:
  - char *p;
  - p = "abc";

  - This assignment makes p point to the first character of the string.

Operations on String Literals

- String literals can be subscripted:
  - char ch;
  - ch = "abc"[1]; //ch is 'b'

- Example
  - For an input number, the corresponding hex digit is returned.

```
char digit_to_hex_char(int digit) {
    return "0123456789ABCDEF"[digit];
}
```

String Literals vs. Character Constants

- A string literal containing a single character is not the same as a character constant.
  - “a” - represented by a pointer.
  - ‘a’ - represented by an integer.

- Examples
  - printf("\n"); //legal
  - printf('\n'); //illegal

String Variables

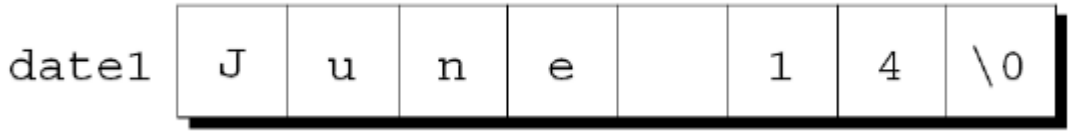
- Any one-dimensional array of characters can be used to store a string.
  - A string must be terminated by a null character.

- Example
  - #define STR_LEN 80
  - char str[STR_LEN + 1];

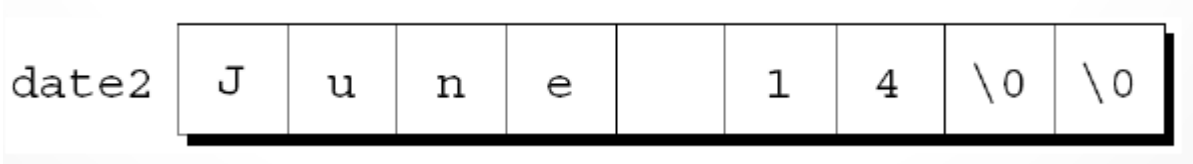
  - Defining a macro that represents 80 and then adding one separately is a common practice.

Initializing a String Variable

- A string variable can be initialized at the time of declaration:
  - char date1[8] = "June 14";//**not a string literal. Array initializer.**
  - char date4[] = "June 14";



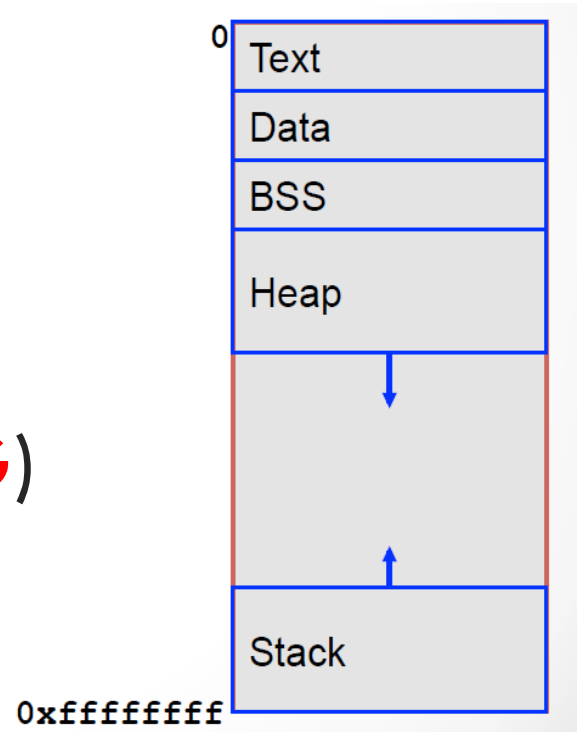
  - char date2[9] = "June 14";



Character Arrays vs Character Pointers

- char date[] = "June 14";
  - declares date to be an array.
  - can be modified, e.g., data[0] = ‘a’;
    - stored in the stack section.

- char *date = "June 14";
  - declares date to be a pointer.
  - cannot be modified, e.g., data[0] = ‘a’; (WRONG)
    - stored in the text section.



Character Arrays vs Character Pointers

- char *p;
  - Using an uninitialized pointer variable as a string makes an error.
    - char *p;
    - p[0] = 'd'; /*** WRONG ***/

- Before using p for a string, you need to set p to point to an arr.
  - char str[STR_LEN+1]="abc", *p;
  - p = str;

  - Q. Is p[0] = 'd'; now valid?

Reading and Writing Strings

- Commonly used functions
  - For writing: printf, puts
  - For reading: scanf, gets

Writing Function: printf

- printf
  - %s conversion for writing a string
  - printf writes the characters in a string one by one until it encounters a null character.

- Example
  - char str[] = "Are we having fun yet?";
  - printf("%s\n", str);

Writing Function: printf

- A conversion specification: %m.ps
  - The first p characters are displayed in a field of size m.
  - If the string has fewer than m characters, it will be right-justified within the field.
    - To force left justification instead, we can put a minus sign in front of m.

- Example
  - printf("%5.2s", "Happy Birthday");

Writing Function: printf

```
int main() {
    char s[] = "01234";
    char* p;
    p = s;
    printf("%c₩n", s[0]);
    printf("%c₩n", *s);
    printf("%c₩n", *(p + 1));
    printf("%s₩n", s);
    printf("%s₩n", p + 1);
}
```

- %d, %c, %f: display the given value
- %s: display characters from the specified address until '\0'

Writing Function: printf

- Displaying substrings

```
int main() {
    char s[] = "01234";
    char* p;
    p = s;
    printf("%s₩n", s);
    printf("%s₩n", p);
    printf("%s₩n", s + 0);
    printf("%s₩n", &(s[0]));
    printf("%s₩n", s + 2);
    printf("%s₩n", &(s[2]));
}
```

Writing Function: printf

- Displaying Characters of a String

```
int main() {
    char s[] = "01234";
    char* p;
    p = s;
    printf("%c₩n", s[0]);
    printf("%c₩n", *p);
    printf("%c₩n", *(p + 0));
    printf("%c₩n", s[2]);
    printf("%c₩n", *(p + 2));
}
```

Writing Function: puts

- puts(str)
  - After writing, puts always adds '\n' to output.

```
#define BUFLEN 200

int main() {
    char buf[BUFLEN];
    gets(buf);
    puts(buf);
    return 0;
}
```

  - equivalent to printf("%s\n", buf);

Reading Function: scanf

- scanf
  - %s conversion: read a string into a character array
  - always stores a null character at the end of the string.

- scanf("%s", str); //str is a pointer.
  - reads characters and stores them in str until it encounters **a white-space character.**

Reading Function: gets

- gets: read an entire line of input

- Properties of gets
  - Does not skip white space before starting to read input.
  - Reads until it finds a new-line character.
  - Discards the new-line character instead of storing it; the null character takes its place.

Reading Function: scanf vs. gets

- Consider the following program fragment:
  - char sentence[SENT_LEN+1];
  - printf("Enter a sentence:\n");
  - scanf("%s", sentence);
  - User enters the following:
    - To C, or not to C: that is the question.

- scanf: will store the string "To"
- gets: will store "To C, or not to C: that is the question."

Reading Function: scanf vs. gets

- Note
  - scanf and gets basically have no way to detect when it's full.
  - Consequently, they may store characters past the end of the array, causing undefined behavior.

- Recommendations
  - scanf: use the conversion specification %ns instead of %s.
    - n: the maximum number of characters
  - gets is inherently unsafe; fgets is a much better alternative.
    - fgets: read from a file.

Accessing the Characters in a String

- A function that counts the number of spaces in a string:

```
int count_spaces(const char s[])
{
    int count = 0, i;
    for (i = 0; s[i] != '₩0'; i++)
        if (s[i] == ' ')
            count++;

    return count;
}
```

Accessing the Characters in a String

- A version that uses pointer arithmetic instead of array subscripting:

```
int count_spaces(const char* s)
{
    int count = 0;
    for (; *s != '₩0'; s++)
        if (*s == ' ')
            count++;
    return count;
}
```

Arrays of Strings

- Array of strings → two-dimensional character array, with one string per row:
  - char planets[][8] = {"Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto"};

- Only the number of rows (i.e., the first dim) can be omitted
  - Not the number of columns

Arrays of Strings

- Unfortunately,
  array planets contains
  a fair bit of wasted space
  (extra null characters):

	0	1	2	3	4	5	6	7
0	M	e	r	c	u	r	y	\0
1	V	e	n	u	s	\0	\0	\0
2	E	a	r	t	h	\0	\0	\0
3	M	a	r	s	\0	\0	\0	\0
4	J	u	p	i	t	e	r	\0
5	S	a	t	u	r	n	\0	\0
6	U	r	a	n	u	s	\0	\0
7	N	e	p	t	u	n	e	\0
8	P	l	u	t	o	\0	\0	\0

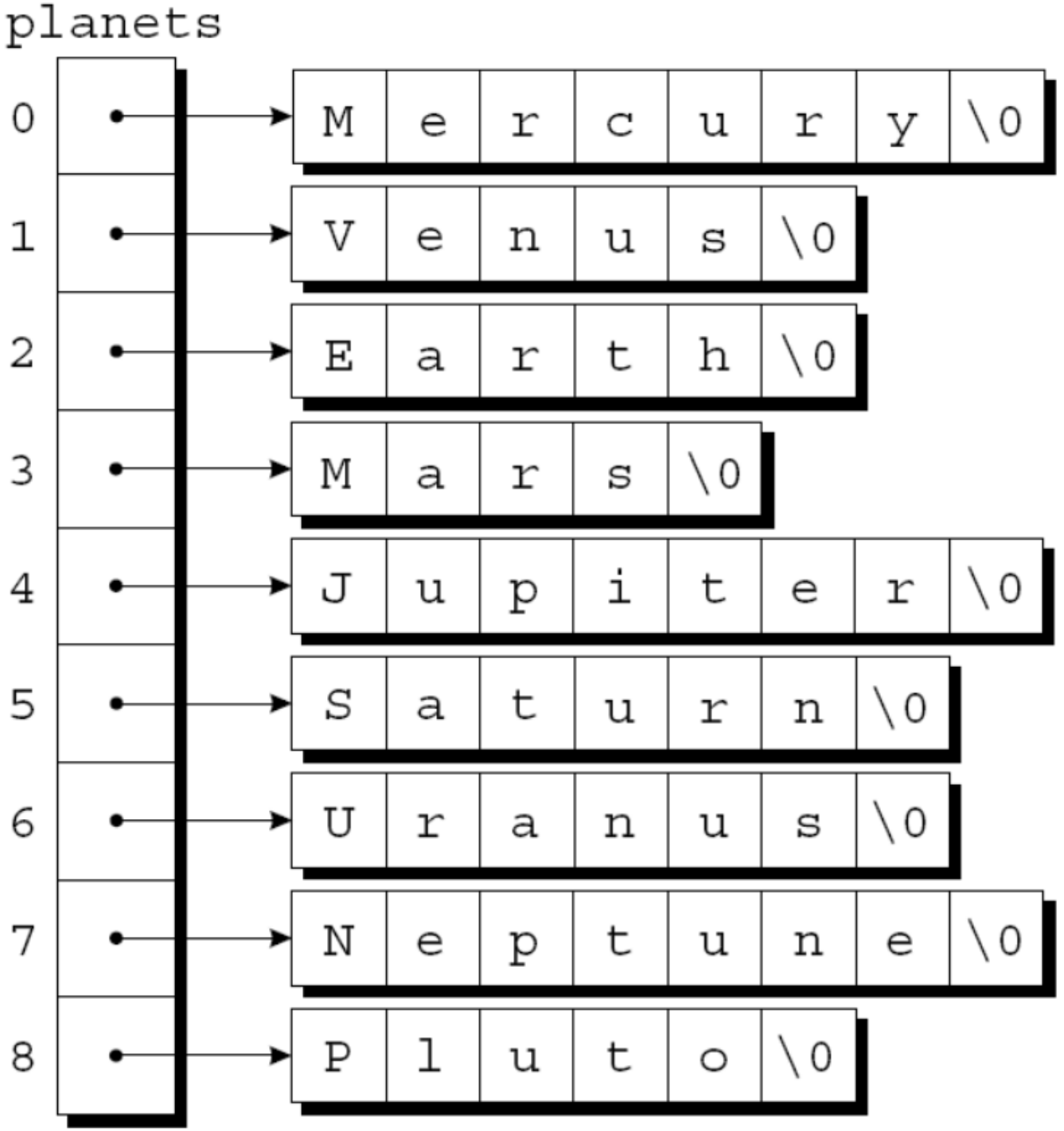
Arrays of Strings

- In a ragged array, rows can have different lengths.
  - Most collections of strings will have a mixture of long strings and short strings.

- We can simulate a ragged array in C by creating an array whose elements are pointers to strings:
  - char *planets[] = {"Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto"};

Arrays of Strings

- This small change has a dramatic effect on how planets is stored:



Arrays of Strings

- To access one of the planet names, all we need to do is subscript the planets array.

- A loop that searches the planets array for strings beginning with the letter M:

```
for (i = 0; i < 9; i++)
  if (planets[i][0] == 'M')
    printf("%s begins with M₩n", planets[i]);
```

Command-Line Arguments

- Available through two parameters to main
  - main(int argc, char *argv[])
  - argc – argument count, i.e. number of args
  - argv – an array of pointers to the arguments

Command-Line Arguments

- Use an integer variable as an index into the argv array to access command-line arguments:

```
for (i = 1; i < argc; i++)
   printf("%s₩n", argv[i]);
```

- Set up a pointer to argv[1], then increment the pointer repeatedly:

```
char** p;
for (p = &argv[1]; *p != NULL; p++)
   printf("%s₩n", *p);
```

String Library Functions

The C String Library

- We want to manipulate strings freely in our program.
  - copy, compare, find, replace, …
  - Since strings are arrays, it is not easy to manipulate them.
  - Example1. want to copy str2 to str1.
    - str1 = str2; → not valid
  - Example2. want to check if str1 and str2 are identical.
    - if (str1==str2) → wrong: it just compares their addresses.

- The C Standard Library provides useful functions for this.

The C String Library

- Character-Handling Library → ctype.h
  - Tests and manipulations of character data
  - ex) Is a character c a digit?
- String-Handling Library → string.h
  - ex) “abc” + “def” = “abcdef”
- String-Conversion Library → stdlib.h
  - String of digits ←→ digits
  - ex) “123” → 123
- Std I/O Library → stdio.h

Character-Handling Library

- ctype.h

Prototype	Function description
`int isdigit( int c );`	Returns a true value if `c` is a digit and 0 (false) otherwise.
`int isalpha( int c );`	Returns a true value if `c` is a letter and 0 otherwise.
`int isalnum( int c );`	Returns a true value if `c` is a digit or a letter and 0 otherwise.
`int isxdigit( int c );`	Returns a true value if `c` is a hexadecimal digit character and 0 otherwise. (See Appendix C, Number Systems, for a detailed explanation of binary numbers, octal numbers, decimal numbers and hexadecimal numbers.)
`int islower( int c );`	Returns a true value if `c` is a lowercase letter and 0 otherwise.
`int isupper( int c );`	Returns a true value if `c` is an uppercase letter and 0 otherwise.
`int tolower( int c );`	If `c` is an uppercase letter, `tolower` returns `c` as a lowercase letter. Otherwise, `tolower` returns the argument unchanged.
`int toupper( int c );`	If `c` is a lowercase letter, `toupper` returns `c` as an uppercase letter. Otherwise, `toupper` returns the argument unchanged.
`int isspace( int c );`	Returns a true value if `c` is a white-space character—newline (`'\n'`), space (`' '`), form feed (`'\f'`), carriage return (`'\r'`), horizontal tab (`'\t'`) or vertical tab (`'\v'`)—and 0 otherwise.

Fig. 8.1 | Character-handling library (`<ctype.h>`) functions. (Part 1 of 2.)

Character-Handling Library

- ctype.h

Prototype	Function description
`int iscntrl( int c );`	Returns a true value if c is a control character and 0 otherwise.
`int ispunct( int c );`	Returns a true value if c is a printing character other than a space, a digit, or a letter and returns 0 otherwise.
`int isprint( int c );`	Returns a true value if c is a printing character including a space (' ') and returns 0 otherwise.
`int isgraph( int c );`	Returns a true value if c is a printing character other than a space (' ') and returns 0 otherwise.

Fig. 8.1 | Character-handling library (<ctype.h>) functions. (Part 2 of 2.)

Example: isdigit

- If false, it returns a zero.
- But, even if true, it may not return 1; it returns just a non-zero value.

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    char c;
    c = '5';
    printf("Result: %d", isdigit(c));

    c = '+';
    printf("₩nResult: %d", isdigit(c));

    return 0;
}
```

Example: islower and toupper

```
int main()
{
    char c;
    c = 'a';
    printf("Result: %d", islower(c));
    c = toupper(c);

    printf("₩nResult: %d", islower(c));

    return 0;
}
```

String-Handling Library

- stdlib.h

Function prototype	Function description
`double atof( const char *nPtr );`	Converts the string `nPtr` to `double`.
`int atoi( const char *nPtr );`	Converts the string `nPtr` to `int`.
`long atol( const char *nPtr );`	Converts the string `nPtr` to `long int`.
`double strtod( const char *nPtr, char **endPtr );`	Converts the string `nPtr` to `double`.
`long strtol( const char *nPtr, char **endPtr, int base );`	Converts the string `nPtr` to `long`.
`unsigned long strtoul( const char *nPtr, char **endPtr, int base );`	Converts the string `nPtr` to `unsigned long`.

Fig. 8.5 | String-conversion functions of the general utilities library.

String-Handling Library

- There are no digit-to-string functions in the C standard.
  - MS compilers include some functions such as itoa and ltoa, but they are not the C standard; it is not guaranteed these functions work in other systems such as linux.
  - Instead, it is recommended to use sprintf.

Function prototype	Function description
`double atof( const char *nPtr );`	Converts the string `nPtr` to `double`.
`int atoi( const char *nPtr );`	Converts the string `nPtr` to `int`.
`long atol( const char *nPtr );`	Converts the string `nPtr` to `long int`.
`double strtod( const char *nPtr, char **endPtr );`	Converts the string `nPtr` to `double`.
`long strtol( const char *nPtr, char **endPtr, int base );`	Converts the string `nPtr` to `long`.
`unsigned long strtoul( const char *nPtr, char **endPtr, int base );`	Converts the string `nPtr` to `unsigned long`.

Fig. 8.5 | String-conversion functions of the general utilities library.

Example: atof

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    const char* num = "90.0";
    double d = atof(num);

    printf("%f", d);

    return 0;
}
```

Example: using sprintf instead of itoa

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num = 100;
    char str1[10] = { 0, };
    char str2[10] = { 0, };

    // Note: itoa is available only in MS C compilers.
    itoa(num, str1, 10);
    printf("itoa: %s₩n", str1);

    sprintf(str2, "%d", num);
    printf("sprintf: %s₩n", str2);

    return 0;
}
```

Std I/O Library

- stdio.h

Function prototype	Function description
`int getchar( void );`	Inputs the next character from the standard input and returns it as an integer.
`char *fgets( char *s, int n, FILE *stream);`	Inputs characters from the specified stream into the array `s` until a newline or end-of-file character is encountered, or until `n - 1` bytes are read. In this chapter, we specify the stream as `stdin`—the standard input stream, which is typically used to read characters from the keyboard. A terminating null character is appended to the array. Returns the string that was read into `s`.
`int putchar( int c );`	Prints the character stored in `c` and returns it as an integer.
`int puts( const char *s );`	Prints the string `s` followed by a newline character. Returns a non-zero integer if successful, or `EOF` if an error occurs.
`int sprintf( char *s, const char *format, ... );`	Equivalent to `printf`, except the output is stored in the array `s` instead of printed on the screen. Returns the number of characters written to `s`, or `EOF` if an error occurs.
`int sscanf( char *s, const char *format, ... );`	Equivalent to `scanf`, except the input is read from the array `s` rather than from the keyboard. Returns the number of items successfully read by the function, or `EOF` if an error occurs.

Example: sprintf

```c
#include <stdio.h>

int main()
{

    char s[80];
    int x;
    double y;
    scanf("%d%lf", &x, &y);
    sprintf(s, "int:%d double:%f", x, y);
    printf("new format: %s", s);

    return 0;
}
```

Example: sscanf

```c
#include <stdio.h>

int main()
{

    char s[] = "31298 87.375";
    int x;
    double y;

    sscanf(s, "%d%lf", &x, &y);
    printf("int: %d₩n", x);
    printf("double: %f₩n", y);

    return 0;
}
```

String-Handling Library

- String manipulations
- string.h

Function prototype	Function description
`char *strcpy( char *s1, const char *s2 )`	Copies string `s2` into array `s1`. The value of `s1` is returned.
`char *strncpy( char *s1, const char *s2, size_t n )`	Copies at most `n` characters of string `s2` into array `s1`. The value of `s1` is returned.
`char *strcat( char *s1, const char *s2 )`	Appends string `s2` to array `s1`. The first character of `s2` overwrites the terminating null character of `s1`. The value of `s1` is returned.
`char *strncat( char *s1, const char *s2, size_t n )`	Appends at most `n` characters of string `s2` to array `s1`. The first character of `s2` overwrites the terminating null character of `s1`. The value of `s1` is returned.

Fig. 8.17 | String-manipulation functions of the string-handling library.

Example: strcpy and strncpy

```c
#include <stdio.h>

int main()
{
    char x[] = "Happy Birthday to You";
    char y[25];
    char z[15];

    strcpy(y, x);
    printf("y: %s₩n", y);

    //copy first 14 chars.
    strncpy(z, x, 14);
    z[14] = '₩0';   //null was not copied.
    printf("z: %s₩n", z);

    return 0;
}
```

More on strcpy: Implementation (1)

```
char* strcpy(char to[], char from[]) {
    int i;
    for (i = 0; from[i] != '₩0'; i++)
        to[i] = from[i];
    to[i] = '₩0' ;
    return to;
}
```

More on strcpy: Implementation (2)

```
char* strcpy(char* to, char* from) {
    char* tmp = to;
    while ((*to = *from) != '₩0') {
        to++; from++;
    }
    return tmp;
}
```

More on strcpy: Implementation (3)

```
char* strcpy(char* to, char* from) {
    char* tmp = to;
    while ((*to++ = *from++) != '₩0' );
    return tmp;
}
```

More on strcpy: Implementation (4)

```
char* strcpy(char* to, char* from) {
    char* tmp = to;
    while (*to++ = *from++);

    return tmp;
}
```

Example: strcat

```c
#include <stdio.h>

int main()
{

    char s1[20] = "Happy ";
    char s2[] = "New Year ";
    char s3[40] = "";

    printf("%s₩n", strcat(s1, s2));
    printf("%s₩n", strncat(s3, s1, 6));

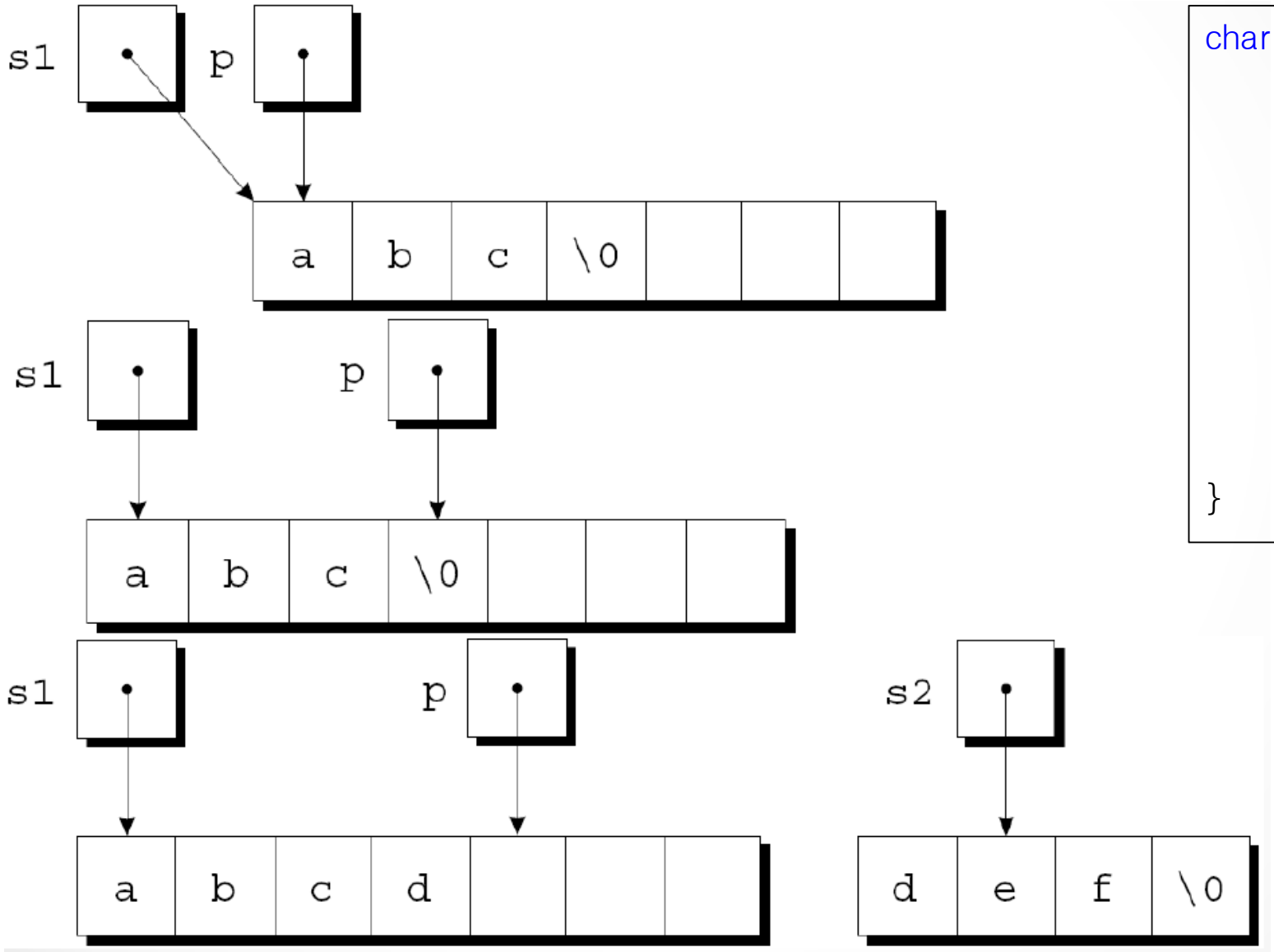


    return 0;
}
```

More on strcat: Implementation

```
char* strcat(char* s1, const char* s2) {
    char* p = s1;
    while (*p != '₩0' )
        p++;
    while (*s2 != '₩0') {
        *p = *s2;
        p++;
        s2++;
    }
    *p = '₩0' ;
    return s1;
}
```

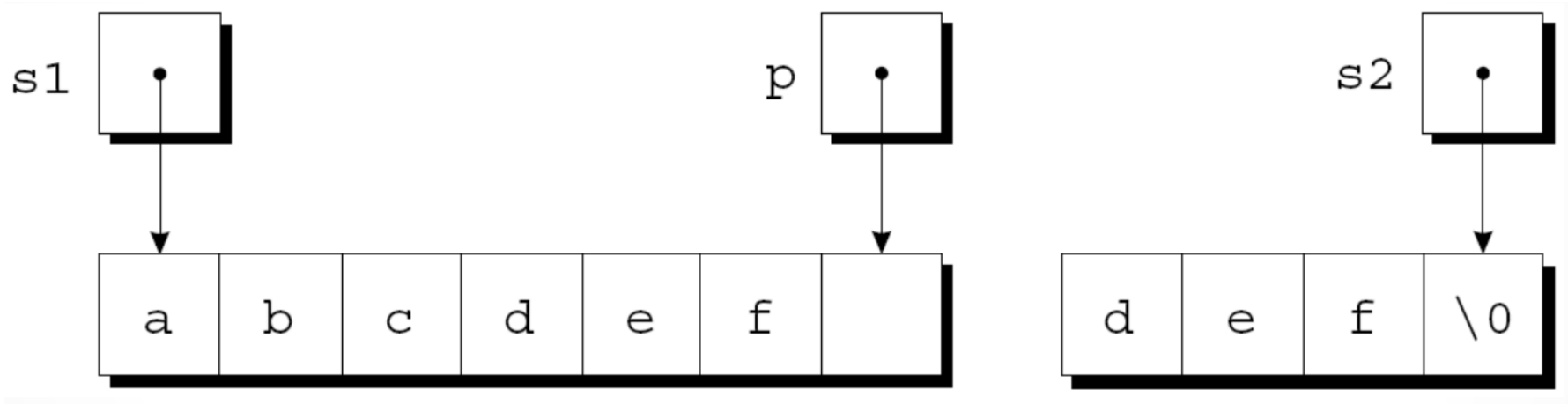
More on strcat: Implementation




```
char* strcat(char* s1, const char* s2) {
     char* p = s1;
     while (*p != '₩0' )
          p++;
     while (*s2 != '₩0') {
          *p = *s2;
          p++;
          s2++;
     }
     *p = '₩0' ;
     return s1;
}
```

More on strcat: Implementation

- The loop terminates when s2 reaches the null character:



- After putting a null character where p points to, strcat returns.

strcat: Condensed Version

- What causes the loop to terminate?

```
char* strcat(char* s1, const char* s2)
{
    char* p = s1;
    while (*p)
        p++;
    while (*p++ = *s2++)
        ;

    return s1;
}
```

Other Library Functions

- n functions
  - char *strncpy(char *t, const char *s, size_t n)
  - char *strncat(char *t, const char *s, size_t n)
  - int strncmp(const char *s1, const char *s2, size_t n)
    - Same as the none-n functions, only works on n chars

- A safer way to use strncpy:
  - strncpy(str1, str2, sizeof(str1) - 1);
  - str1[sizeof(str1)-1] = '\0';

String-Handling Library

- String comparisons
- string.h

Function prototype	Function description
`int strcmp( const char *s1, const char *s2 );`	Compares the string `s1` with the string `s2`. The function returns `0`, less than `0` or greater than `0` if `s1` is equal to, less than or greater than `s2`, respectively.
`int strncmp( const char *s1, const char *s2, size_t n );`	Compares up to `n` characters of the string `s1` with the string `s2`. The function returns `0`, less than `0` or greater than `0` if `s1` is equal to, less than or greater than `s2`, respectively.

Fig. 8.20 | String-comparison functions of the string-handling library.

Example

```c
#include <stdio.h>
#include <string.h>

int main()
{

    const char* s1 = "Happy New Year";
    const char* s2 = "Happy New Year";
    const char* s3 = "Happy Birthday";

    printf("s1==s2 ? %d₩n", strcmp(s1, s2));
    printf("s1==s3 ? %d₩n", strcmp(s1, s3));
    printf("s1==s3 up to 3 ? %d₩n", strncmp(s1, s3, 3));
    printf("s1==s3 up to 7 ? %d₩n", strncmp(s1, s3, 7));

    return 0;
}
```

More on strcmp

- Cannot compare strings using ==

```
int strcmp(char* s, char* t) {
   for (; *s == *t; s++, t++) {
      if (*s == '₩0' )
         return 0;
   }

   return *s - *t;
}
```

- Returns 0 if s == t
- otherwise, returns the difference between the first different chars

String-Handling Library

- Search
- string.h

Function prototype and description
`char *strchr( const char *s, int c );` Locates the first occurrence of character `c` in string `s`. If `c` is found, a pointer to `c` in `s` is returned. Otherwise, a `NULL` pointer is returned.
`size_t strcspn( const char *s1, const char *s2 );` Determines and returns the length of the initial segment of string `s1` consisting of characters *not* contained in string `s2`.
`size_t strspn( const char *s1, const char *s2 );` Determines and returns the length of the initial segment of string `s1` consisting only of characters contained in string `s2`.
`char *strpbrk( const char *s1, const char *s2 );` Locates the first occurrence in string `s1` of any character in string `s2`. If a character from string `s2` is found, a pointer to the character in string `s1` is returned. Otherwise, a `NULL` pointer is returned.
`char *strrchr( const char *s, int c );` Locates the last occurrence of `c` in string `s`. If `c` is found, a pointer to `c` in string `s` is returned. Otherwise, a `NULL` pointer is returned.
`char *strstr( const char *s1, const char *s2 );` Locates the first occurrence in string `s1` of string `s2`. If the string is found, a pointer to the string in `s1` is returned. Otherwise, a `NULL` pointer is returned.
`char *strtok( char *s1, const char *s2 );` A sequence of calls to `strtok` breaks string `s1` into "tokens"—logical pieces such as words in a line of text—separated by characters contained in string `s2`. The first call contains `s1` as the first argument, and subsequent calls to continue tokenizing the same string contain `NULL` as the first argument. A pointer to the current token is returned by each call. If there are no more tokens when the function is called, `NULL` is returned.

Example: strchr

```c
#include <stdio.h>
#include <string.h>

int main()
{
    const char* string = "This is a test";
    char c1 = 'z';

    if (strchr(string, c1) != NULL) {
        printf("Found %c", c1);
    }
    else {
        printf("Not Found %c", c1);
    }

    return 0;
}
```

Example: strstr

```c
#include <stdio.h>
#include <string.h>

int main()
{
    const char* str1 = "abcdefabcdef";
    const char* str2 = "def";

    printf("%s", strstr(str1, str2));

    return 0;
}
```

Example: strstr

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char string[] = "This is a sentence with 7 tokens";
    char* tokenPtr;

    tokenPtr = strtok(string, " ");

    while (tokenPtr != NULL) {
        printf("%s₩n", tokenPtr);
        tokenPtr = strtok(NULL, " ");
    }

    return 0;
}
```

String-Handling Library

- Memory Management
- string.h

Function prototype	Function description
`void *memcpy( void *s1, const void *s2, size_t n );`	Copies `n` characters from the object pointed to by `s2` into the object pointed to by `s1`. A pointer to the resulting object is returned.
`void *memmove( void *s1, const void *s2, size_t n );`	Copies `n` characters from the object pointed to by `s2` into the object pointed to by `s1`. The copy is performed as if the characters were first copied from the object pointed to by `s2` into a temporary array and then from the temporary array into the object pointed to by `s1`. A pointer to the resulting object is returned.
`int memcmp( const void *s1, const void *s2, size_t n );`	Compares the first `n` characters of the objects pointed to by `s1` and `s2`. The function returns `0`, less than `0` or greater than `0` if `s1` is equal to, less than or greater than `s2`.
`void *memchr( const void *s, int c, size_t n );`	Locates the first occurrence of `c` (converted to `unsigned char`) in the first `n` characters of the object pointed to by `s`. If `c` is found, a pointer to `c` in the object is returned. Otherwise, `NULL` is returned.
`void *memset( void *s, int c, size_t n );`	Copies `c` (converted to `unsigned char`) into the first `n` characters of the object pointed to by `s`. A pointer to the result is returned.

Example: memcpy

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[17];
    char s2[] = "Copy this string";

    memcpy(s1, s2, 17);
    printf("%s", s2);


    return 0;
}
```

Example: memmove

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char x[] = "Home Sweet Home";
    memmove(x, &x[5], 10);
    printf("%s", x);

    return 0;
}
```

Example: memcmp

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[] = "ABCDEFG";
    char s2[] = "ABCDXYZ";

    printf("s1 == s2 up to 4? %d₩n", memcmp(s1, s2, 4));
    printf("s1 == s2 up to 7? %d₩n", memcmp(s1, s2, 7));
    printf("s2 == s1 up to 7? %d₩n", memcmp(s2, s1, 7));

    return 0;
}
```

Example: memset

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char string[] = "BBBBBBBBBB";

    printf("%s", memset(string, 'b', 7));

    return 0;
}
```

String-Handling Library

- Others
- string.h

Function prototype	Function description
`char *strerror( int errornum );`	Maps `errornum` into a full text string in a compiler- and locale-specific manner (e.g. the message may appear in different languages based on its location). A pointer to the string is returned.
`size_t strlen( const char *s );`	Determines the length of string `s`. The number of characters preceding the terminating null character is returned.

```c
#include <stdio.h>
#include <string.h>

int main()
{
    printf("[err1] %s₩n", strerror(1));
    printf("[err2] %s₩n", strerror(2));
    printf("[err3] %s₩n", strerror(3));
    return 0;
}
```

String-Handling Library: strlen

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "ann"; char s2[] = "abby";
    char s3[20];
    printf("%d₩n", strlen(s));
    printf("%d₩n", strlen(&s[1]));
    strcpy(s3, s);
    strcat(s3, s2);
    printf("%s₩n", &s3[2]);
    printf("%d₩n", strcmp(s, s2));
    return 0;
}
```

strlen implementation (1)

```
size_t strlen(const char* s) {
    size_t n;
    for (n = 0; *s != '₩0'; s++)
        n++;
    return n;
}
```

strlen implementation (2)

```
size_t strlen(const char* s) {
    size_t n = 0;
    for (; *s != '₩0'; s++)
        n++;
    return n;
}
```

strlen implementation (3)

```
size_t strlen(const char* s) {
    size_t n = 0;
    for (; *s; s++)
        n++;
    return n;
}
```

strlen implementation (4)

```
size_t strlen(const char* s) {
    size_t n = 0;
    for (; *s++; )
        n++;
    return n;
}
```

strlen implementation (5)

```
size_t strlen(const char* s) {
    size_t n = 0;
    while (*s++)
        n++;
    return n;
}
```

strlen implementation (6)

```
size_t strlen(const char* s) {
    const char* p = s;
    while (*s)
        s++;
    return s - p;
}
```

Searching for the End of a String

```
while (*s) s++;
```

- Leaves s pointing to the null character.

```
while (*s++);
```

- More concise, but leaves s pointing just past the null character.

References

- Chapter 8. C Characters and Strings, C: How to Program, Paul Deitel et al.